

LSTM Model Fine-Tuning & Hyperparameter Exploration using NLTK Gutenberg (Shakespeare's Text)

Objective

The objective of this project is to develop and fine-tune a **Long Short-Term Memory (LSTM)** model capable of learning the **writing style, structure, and word patterns** of Shakespeare's *Hamlet* text.

The trained model should then **generate new sentences** that sound similar to Shakespeare's writing style.

Concept Overview

- **LSTM (Long Short-Term Memory)** is a type of Recurrent Neural Network (RNN) designed to handle sequential data and remember long-term dependencies.
- It is ideal for tasks like **text prediction, language modeling, and speech processing**.
- Here, the LSTM model learns from large text data (Shakespeare's *Hamlet*) and predicts the **next word** in a sentence based on previous words.

Methodology

1. Data Collection

- Dataset: *Hamlet* text from **NLTK's Gutenberg corpus**.
- Text is cleaned by removing punctuation, converting to lowercase, and formatting sentences.

2. Text Preprocessing

- **Tokenization:** Each word is converted into a numerical token using Keras Tokenizer.
- **Sequence Generation:** From every sentence, n-gram sequences are created (e.g., "to be or not to" → next word "speak").
- **Padding:** Sequences are padded to ensure uniform input length.

3. Model Architecture

Layer	Description
Embedding Layer	Converts words into dense vector representations.
LSTM Layer	Learns contextual word relationships and remembers prior words.
Dense (Softmax) Layer	Outputs probabilities for all words in the vocabulary to choose the next word.

4. Hyperparameter Tuning

Used **KerasTuner (RandomSearch)** to find the best parameters:

- Embedding Dimension → {64, 128, 256}
- LSTM Units → {64, 128, 256}
- Dropout → {0.2–0.5}
- Learning Rate → {0.001, 0.0001}
- Batch Size → {32, 64}

5. Model Training

- The tuner automatically trained multiple model configurations.
- The best hyperparameters were selected based on **validation accuracy**.
- The final model was retrained for improved performance.

6. Text Generation

- A **seed text** (e.g., “to be or not to”) is given to the model.
- The model predicts the **next word** by calculating probabilities for every word in its vocabulary and selecting the highest one.
- This process repeats iteratively to generate a full Shakespeare-like sentence.

Results

Metric	Observation
Best Hyperparameters	embedding_dim = 128, lstm_units = 64, learning_rate = 0.001

Metric	Observation
Validation Accuracy	~0.80 (approx., on reduced dataset)
Generated Text Example	“to be or not to speak of the king to make his son”

The generated sentence reflects Shakespeare’s syntax and flow, showing that the LSTM successfully learned contextual word relationships.

Understanding the Prediction Process

1. The model reads the seed sentence “to be or not to”.
2. It compares this pattern with those learned from training data.
3. It assigns probabilities to all possible next words (from the 4,797-word vocabulary).
4. The word with the highest probability (e.g., “*speak*”) is chosen.
5. This new word is appended to the sequence, and the process repeats until the target word count is reached.

Conclusion

This project demonstrates the capability of LSTM networks to **learn language structure** and **generate human-like text** when trained on literary data.

Through hyperparameter tuning, model performance was optimized for efficient learning and realistic text output.

The experiment showcases a practical application of **deep learning in Natural Language Processing (NLP)** and provides hands-on understanding of how AI models like LSTM learn, remember, and create new text sequences.